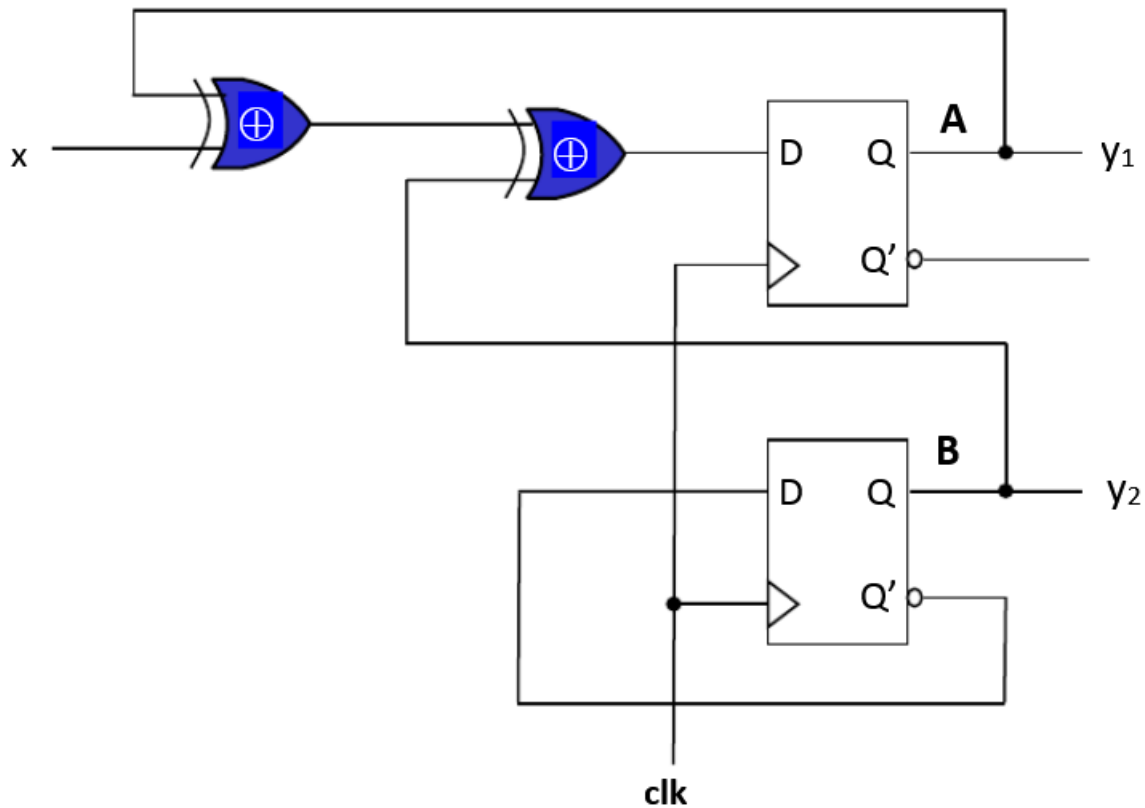
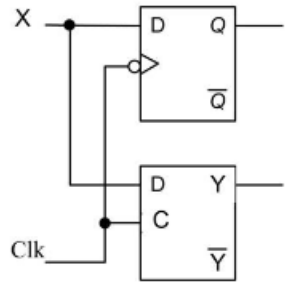
a. Obtain the next state and output equations, and the state table.

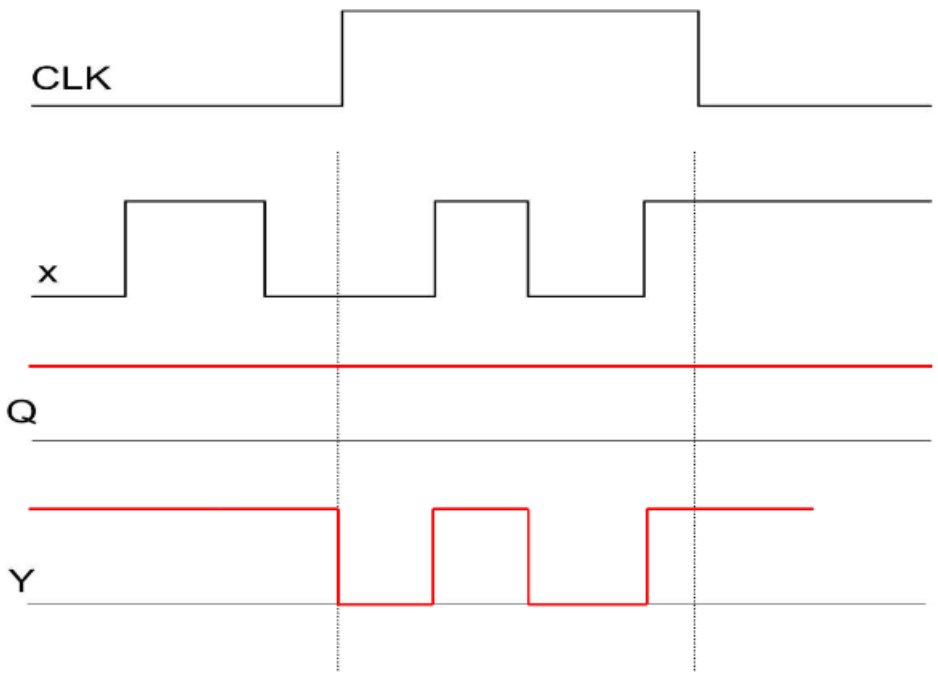
b. Draw the state diagram.

c. What does this circuit do?



II. For the shown D-latch and D-FF, plot the outputs Q and Y for the shown applied signals X and CLK. Assume that the initial state of **both** memory elements to be **1,** and that all delays =0. (**4 points**)





```
module ex50
  ( input logic reset, clock,
    output logic [3:0] count ) ;

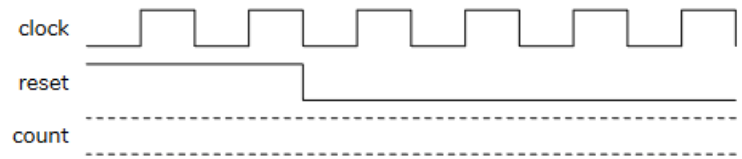
   logic [3:0] count_next ;

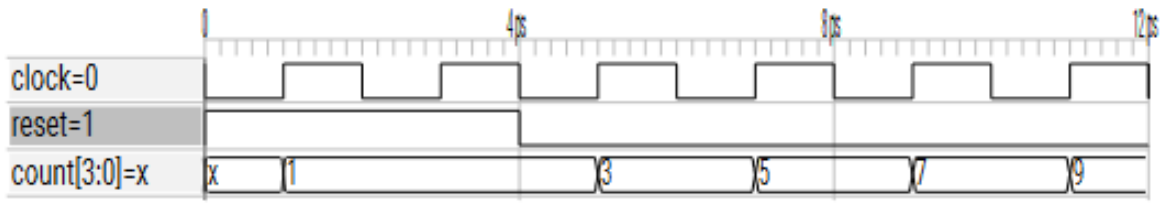
   assign count_next
     = reset ? 4'd1 :
       count + 4'd2 ;

   always_ff @(posedge clock) count = count_next ;

endmodule
```

and the input signals in the diagram below, draw a vertical line between the dashed lines where the **count** output changes and write the value of **count** in-between each vertical line. Write an **x** if the value is undefined.



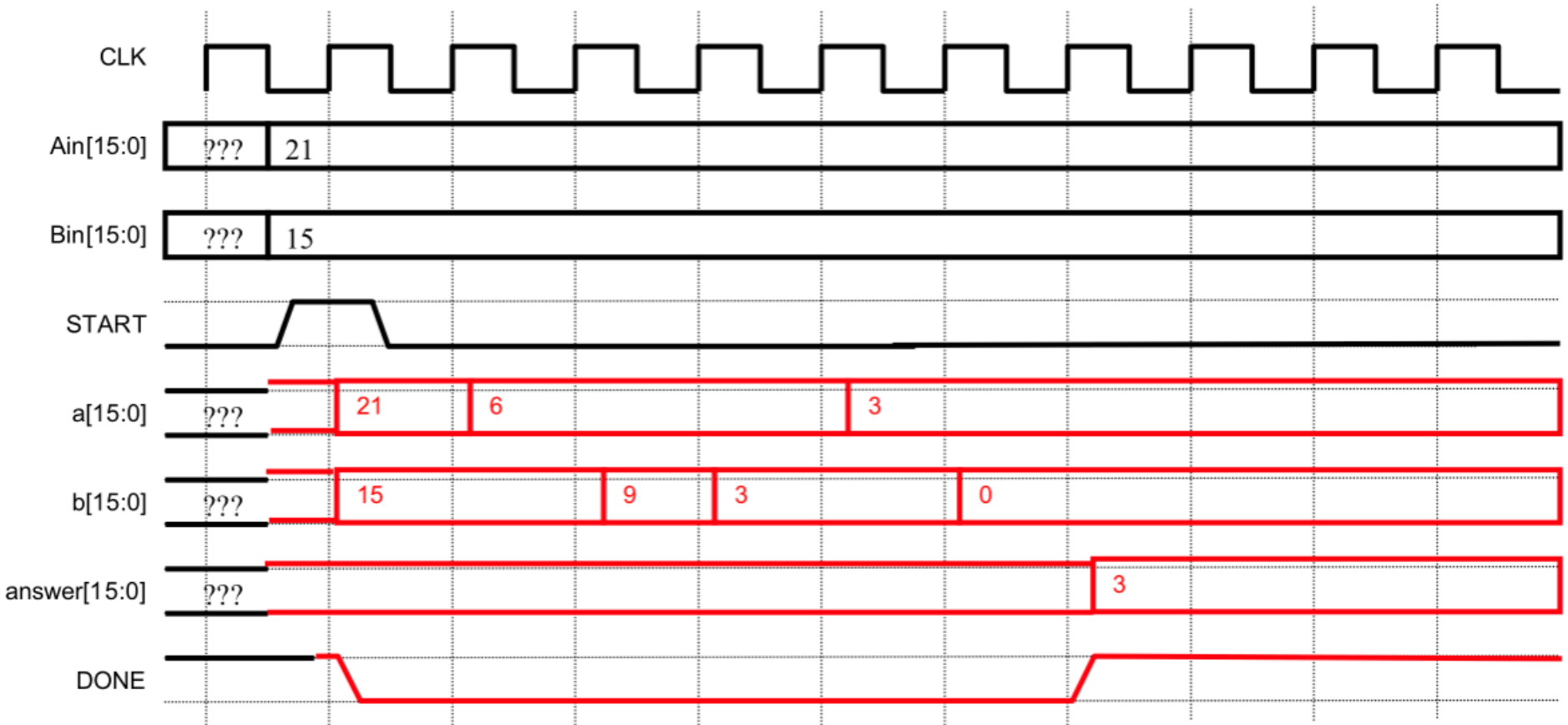


Consider the following Verilog module that uses Euclid's algorithm to iteratively compute the greatest common divisor of two 16-bit unsigned integer values Ain and Bin where Ain ≥ Bin.

```
module gcd(clk,start,Ain,Bin,answer,done);
  input clk,start;
  input [15:0] Ain,Bin;
  output reg [15:0] answer;
  output reg done;

  reg [15:0] a,b;
  always @ (posedge clk) begin
    if (start) begin a <= Ain; b <= Bin; done <= 0; end
    else if (b == 0) begin answer <= a; done <= 1; end
    else if (a > b) a <= a - b;
    else b <= b - a;
  end
endmodule
```

Please neatly complete the timing diagram below as the module computes the gcd of 21 and 15. Use "???" to indicate values that cannot be determined from the information given.


CLK
Ain[15:0]
???
21
Bin[15:0]
???
15
START
a[15:0]
???
21
6
3
b[15:0]
???
15
9
3
0
answer[15:0]
???
3
DONE

i) Given F(a,b,c,d)= (a⊕d)((bc)'), how would F be represented using the sum of products notation?

a) F(a, b, c, d) = ∑(3, 5, 7, 10, 12, 14)

b) F(a, b, c, d) = ∑(0, 2, 4, 9, 11, 13)

c) F(a, b, c, d) = ∑(1, 3, 5, 8, 10, 12)

d) F(a, b, c, d) = ∑(2, 4, 6, 11, 13, 15)

ii) What is the binary value of the wire in the code below after the assignment?

```
wire [7:0] thirty_k_above_atlantic;
assign thirty_k_above_atlantic = {{2{2'b01}}, 3'd1, 1'b0};
```

a. 01010010 b. 01011110 c. 01011000 d. 01001110

iii) Which one of the following Boolean functions is NOT equivalent to the others?

a. F(a, b, c, d) = ((a'b' + c')d')'

b. F(a, b, c, d) = Π(0, 2, 4, 8, 12)

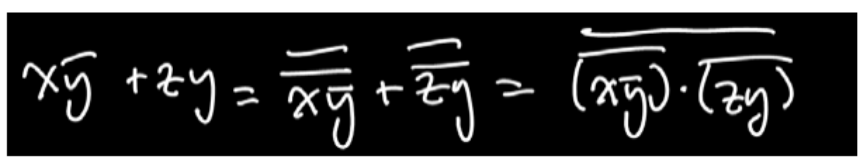
c. F(a, b, c, d) = ac + (c+d)(b+d)

d. F(a, b, c, d) = ac + a'd + bc + bd

e. F(a, b, c, d) = c'd + (a+b)c + cd

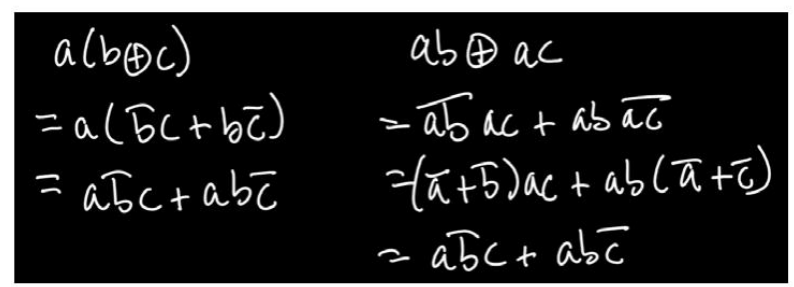
[2] (i). Show that the following equation is correct, using Boolean algebra:

$$x\bar{y} + zy = \overline{\left(\overline{x\bar{y}}\right)\cdot\left(\overline{zy}\right)}$$



[4] (ii). Show that the following equation is correct, using Boolean algebra. Hint: use the fact that $x \oplus y = \bar{x}y + x\bar{y}$ to derive your answer.

$$a(b \oplus c) = ab \oplus ac$$



Q1. You are to find the minimal sum-of-products expression for the following four-input logic function (in which *D* represents a don't care):

$$f(a,b,c,d) = \sum m(0,2,5,7,10,14,15) + \sum D(8,13)$$

Perform your solution in two steps:

(a) Fill out the Karnaugh map below corresponding to this function:

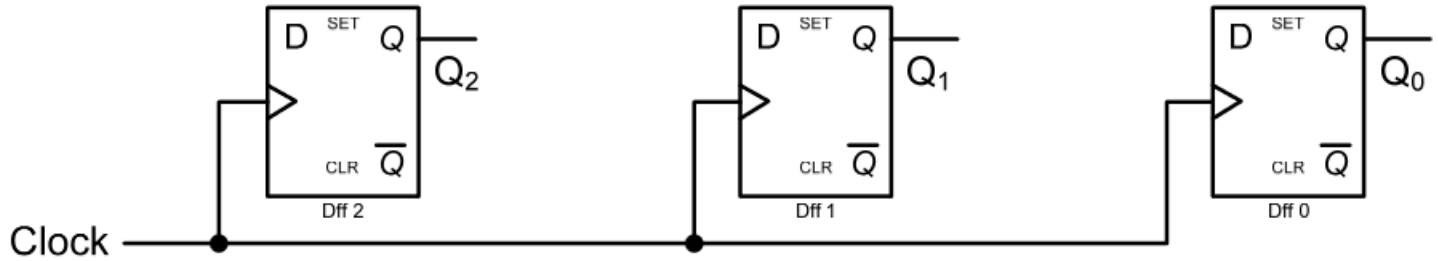
c d \ a b	0 0	0 1	1 1	1 0
0 0				
0 1				
1 1				
1 0				

c d \ a b	0 0	0 1	1 1	1 0
0 0	1	0	0	x
0 1	0	1	x	0
1 1	0	1	1	0
1 0	1	0	1	1

(b) Use the K-map to determine the minimal sum-of-products expression

F = b'd' + abc + bd
Or
F = b'd' + acd' + bd

[10] **Q6**. You are to design a circuit that has three flip-flops, $Q_2Q_1Q_0$, as indicated in the diagram below. The outputs of the flip-flop over consecutive clock cycles should follow the pattern $Q_2Q_1Q_0 =$ 000, 010, 100, 110, 001, 011, 101, 111, 000, and so on (repeats). You can assume that all flip-flops in the circuit are manually reset to 0 to start the pattern.



(i) Fill in the table below, which shows the required values of the D inputs of each flip-flop such that the required pattern will be produced.

Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	0	1	0
0	1	0	1	0	0
1	0	0	1	1	0
1	1	0	0	0	1
0	0	1	0	1	1
0	1	1	1	0	1
1	0	1	1	1	1
1	1	1	0	0	0

(ii) In the space below derive an appropriate logic circuit to connect to each flip-flop D input. use AND, OR, NOT, and XOR operations, but generate the simplest expressions you can.

$Q_2 Q_1$ / Q_0	0 0	0 1	1 1	1 0
0		1		1
1		1		1

$D_2 = \overline{Q_2}Q_1 + Q_2\overline{Q_1} = Q_2 \oplus Q_1$ 2 mar

$Q_2 Q_1$ / Q_0	0 0	0 1	1 1	1 0
0	1			1
1	1			1

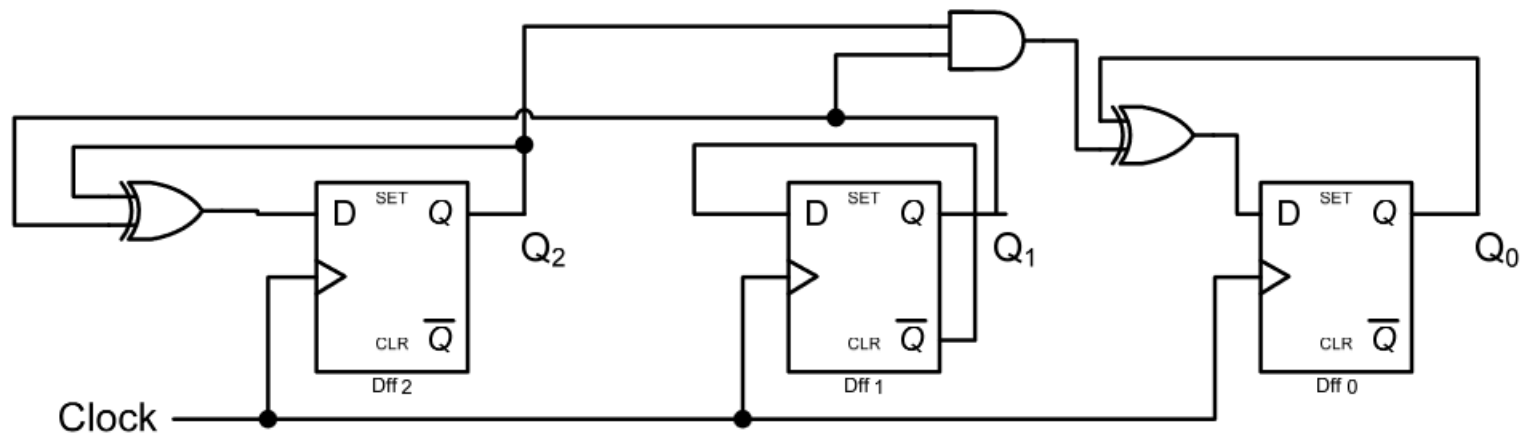
$D_1 = \overline{Q_1}$ 1 ma

$Q_2 Q_1$ / Q_0	0 0	0 1	1 1	1 0
0			1	
1	1	1		1

$D_0 = Q_2Q_1\overline{Q_0} + \overline{Q_2}Q_0 + \overline{Q_1}Q_0$

$= (Q_2Q_1) \oplus Q_0$ 2 ma

(iii) Draw the final circuit diagram that shows the flip-flops and the logic circuits you derived in part (ii).



3) Write a Verilog module called **Sat8bit.** It is to implement an eight-bit *saturating* up counter with enable and reset.
   - Inputs are **clock**, **reset** and **enable**
   - Output is the counter's value, **Q[7:0]**.

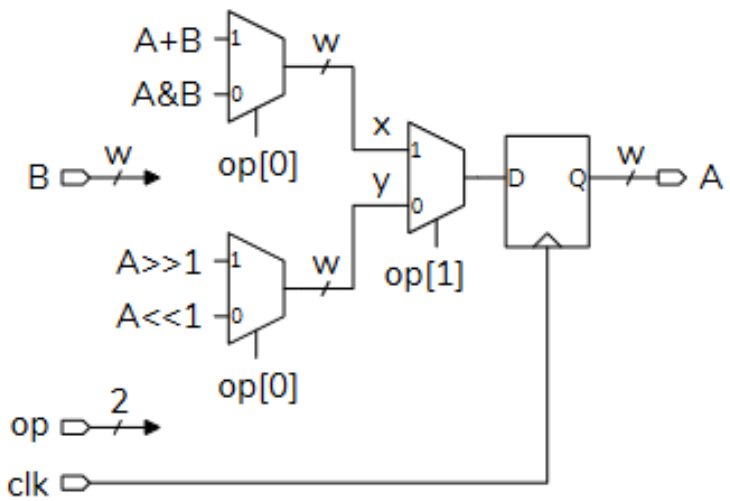
The counter should set its value to 0 if **reset** is asserted on the positive edge of **clock**. If reset is a 0 and enable is a 1 it is to increment its value on the rising edge the **clock.** Otherwise the register should hold its value. You will be graded for correctness, syntax, and efficiency of your design and code. **[9 points]**

```
module Sat8bit(clock, reset, enable, Q);
  input clock;
  input reset;
  input enable;

  output reg [7:0] Q;

  always @(posedge clock) begin
    if (reset) begin
      Q <= 8'h0;
    end
    else if (enable && (Q!=8'hff)) begin
      Q <= Q + 1;
    end
  end
endmodule
```

Write a Verilog module named **alu** corresponding to the diagram shown below. The module has a w-bit input **B**, a 2-bit input **op**, a clock input **clk**, and a w-bit registered output **A**. The module declaration should include a parameter named w for the bus widths with a default value of 16. The use of the signals **x** and **y** is optional but they must be declared if they're used. Bits should be numbered in decreasing order. Follow the course coding conventions. You may omit comments.



Answer

An answer using **x** and **y** is:

```
module alu
  #(parameter w=16)
  ( input logic [w-1:0] B,
    input logic [1:0] op,
    input logic clk,
    output logic [w-1:0] A ) ;

   logic [w-1:0] x, y ;

   assign x = op[0] ? A+B : A&B ;
   assign y = op[0] ? A>>1 : A<<1 ;

   always @(posedge clk) A
     <= op[1] ? x : y ;

endmodule
```

Draw a block diagram for the following Verilog module. Follow the course conventions for block diagrams.

```
module exam
   ( input logic start, clock,
     output logic [15:0] count ) ;

   always_ff @(posedge clock) count
     <= start ? 16'b1 :
        count[15] == 1'b1 ? 16'b1 :
        count << 1 ;

endmodule
```

Answer

